

Smart MCQ Solver Challenge

Project Report

BSDA2001P: Introduction to DL and GenAI Project

Submitted by

Sandhya Gupta

Roll No: 22f3000356

Kaggle Leaderboard Score: 0.75893 | Rank: 237

Contents

1. Abstract
2. Introduction
3. Dataset & Preprocessing
4. Tokenization Strategy
5. Modeling & Experimentation (5.1 ELECTRA, 5.2 Custom Transformer, 5.3 DeBERTa)
6. Performance & Comparative Analysis
7. Conclusion & Future Work
8. References

1. Abstract

This report documents my work on the Kaggle 'Smart MCQ Solver Challenge', predicting the correct answer among five options for academic multiple-choice questions and ranking the top three candidate answers according to their predicted scores/probabilities, scored with Mean Average Precision at 3 (MAP@3). I represented the task as candidate-answer scoring: depending on the model, each question was either expanded into five question–candidate-answer pairs for binary relevance classification or processed jointly as a five-choice multiple-choice input

Three models were built and compared: a fine-tuned ELECTRA-base transformer (Model 1, HuggingFace AutoModelForMultipleChoice), a Transformer encoder implemented entirely from scratch in PyTorch (Model 2 — embeddings, sinusoidal positional encoding, and multi-head self-attention written manually), and a fine-tuned DeBERTa-v3-small model (Model 3). A TF-IDF + Logistic Regression pipeline was also built as an early classical baseline. The final submission blends the ELECTRA and calibrated from-scratch-Transformer probabilities (80%/20%).

On held-out validation data, the from-scratch Transformer reached a 5-fold MAP@3 of 0.9947, DeBERTa-v3-small reached 0.9556 on an 85/15 hold-out, and ELECTRA — evaluated on a stricter, leakage-safe GroupKFold split — reached 0.673. This gap is itself a key finding (Section 6): the final Kaggle leaderboard score of 0.75893 (rank 237) sits much closer to ELECTRA's conservative estimate than to the other two, underlining that validation-split design matters as much as model choice.

2. Introduction

Problem statement. Given train.csv (2,000 labelled MCQs: prompt, options A-E, answer) and test.csv (500 unlabelled MCQs), predict the top-3 most likely correct options per test question, scored by MAP@3. Questions are academic/technical (physics, philosophy, astrophysics) with long, semantically similar distractor options, making surface word-overlap alone insufficient.

Objectives. (1) Build a competitive, submittable solution; (2) demonstrate first-principles understanding of the Transformer architecture by implementing one from scratch; (3) gain hands-on experience fine-tuning pretrained language models with leakage-aware validation and W&B tracking; (4) compare classical ML, a custom deep-learning architecture, and fine-tuned Transformers honestly.

Report structure. Section 3 covers the dataset and preprocessing; Section 4 the tokenization strategy; Section 5 the three required models; Section 6 compares their performance including the leaderboard result; Section 7 reflects on learnings and future work; Section 8 lists references.

3. Dataset & Preprocessing

3.1 Dataset description. The dataset contains 2,000 training questions, with each instance consisting of an id, a question prompt, five candidate answers (A–E), and the correct answer label. The task is to rank the three most likely answers for each question, with performance evaluated using MAP@3.

3.2 EDA. EDA showed that the dataset contained no missing values. The answer distribution was moderately imbalanced, with B being the most frequent correct answer (490), followed by C (459), A (369), D (358), and E (324). Prompt lengths had a mean of 117.67 characters, median of 111, and maximum of 337, while answer-option lengths were also highly variable. Duplicate analysis found 242 repeated prompt occurrences, with 454 rows belonging to duplicate-prompt groups; importantly, no duplicated prompt was associated with conflicting answer labels. These findings highlighted moderate class imbalance, variable text lengths, and the need to consider duplicate questions when designing validation strategies.

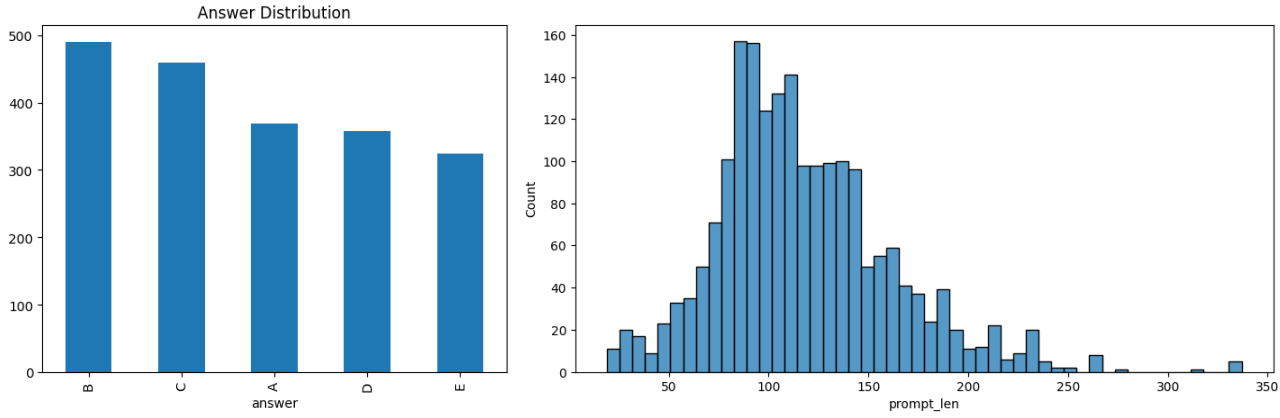


Figure 1 — Answer-letter distribution and prompt-length distribution (train set, $n=2,000$).

3.3 Preprocessing. No missing-value treatment was required because the dataset contained no missing values. For the TF-IDF baseline, question–candidate-answer pairs were converted into unigram and bigram TF-IDF features. For the custom Transformer, text was lowercased, non-alphanumeric characters were replaced with spaces, and a custom vocabulary containing special tokens [PAD], [UNK], [CLS], and [SEP] was constructed. Question and candidate-answer text were encoded together and padded or truncated to the required maximum sequence length. For ELECTRA and DeBERTa, pretrained Hugging Face tokenizers were used to tokenize question–answer pairs with padding, truncation, and attention masks.

3.4 Data augmentation. No artificial data augmentation was applied. Since modifying academic questions or answer choices could change their meaning or alter the correct answer, the original textual data was retained. Model performance was instead improved through different architectures, pretrained models, fine-tuning, validation strategies, and ensemble methods.

4. Tokenization Strategy

Primary tokenizer & justification. The best-performing model in our experiments was the custom Transformer encoder, which achieved a 5-fold validation MAP@3 of 0.9947. For this model, a custom word-level tokenizer was implemented from scratch. The tokenizer converts text to lowercase, removes non-alphanumeric characters, splits the text into individual words, and constructs a vocabulary of up to 30,000 tokens. Special tokens [PAD], [UNK], [CLS], and [SEP] were included to represent padding, unknown words, sequence boundaries, and the separation of the question from the candidate answer. This approach provided a simple and controlled tokenization pipeline suitable for demonstrating the Transformer architecture from scratch.

Implementation. The question and candidate answer were tokenized separately and combined in the format [CLS] question [SEP] candidate answer [SEP]. Each sequence was converted into token IDs and padded or truncated to a maximum length of 256 tokens. An attention mask was generated simultaneously, with 1 representing valid tokens and 0 representing padding. The resulting token IDs and attention masks were converted into PyTorch tensors and supplied to the model through a custom Dataset and DataLoader.

Experimental comparison. Different tokenization approaches were used across the models. The custom Transformer used the manually implemented word-level tokenizer, while the pretrained ELECTRA-base and DeBERTa-v3-small models used their respective pretrained Hugging Face tokenizers. The TF-IDF baseline used word and bigram features instead of neural tokenization. The custom tokenizer was selected for the from-scratch Transformer because it allowed the complete text-processing and Transformer pipeline to be implemented without relying on a pretrained tokenizer, while the pretrained models used their original tokenizers to remain consistent with their pretrained representations.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Transformer — ELECTRA-base

Architecture. Architecture: ELECTRA-base-discriminator is a pretrained Transformer encoder developed by Google. It contains 12 Transformer encoder layers with multi-head self-attention and position-wise feed-forward networks. ELECTRA differs from BERT primarily in its pretraining objective: instead of predicting only masked tokens, it uses replaced-token detection, where the discriminator learns to distinguish original tokens from tokens replaced by a generator. For this project, google/electra-base-discriminator was loaded using Hugging Face's AutoModelForMultipleChoice, allowing the model to process the five candidate answers and produce a score for each option. (Clark et al., 2020, ICLR — <https://arxiv.org/abs/2003.10555>)

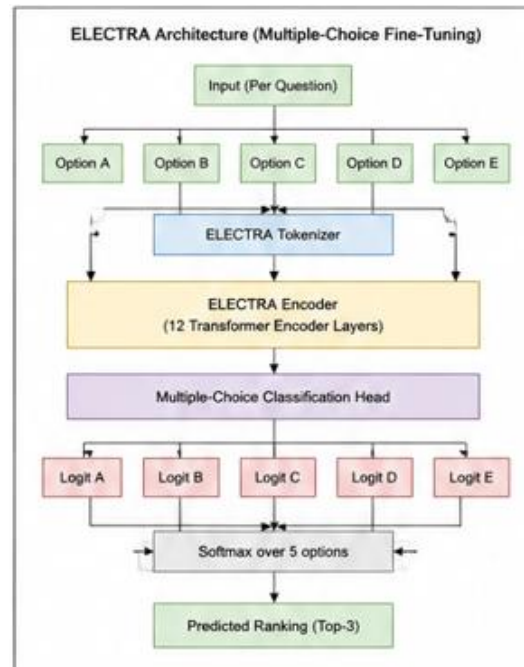


Figure — Architecture of the pretrained ELECTRA Model.

Salient points. ELECTRA uses bidirectional self-attention, allowing the model to capture relationships between different parts of the question and candidate answer. Its pretrained representations are particularly useful for an NLP task such as MCQ solving because the model has already learned general language patterns from large-scale pretraining. The multiple-choice formulation also directly produces scores for the five candidate options, which can then be ranked according to their predicted probabilities.

Fine-tuning strategy. The pretrained ELECTRA model was fine-tuned end-to-end using AutoModelForMultipleChoice. In the implementation, the model used a maximum sequence length of 192, batch size 4 with gradient accumulation over 4 steps, learning rate of 1.5×10^{-5} , weight decay of 0.02, and 5 training epochs. A cosine learning-rate schedule with 10% warm-up was used. No layers were frozen, allowing the pretrained parameters to adapt to the MCQ task. Mixed-precision autocasting and gradient scaling were used during training to reduce GPU memory usage and improve computational efficiency. No model quantization was applied.

5.2 Model 2: Custom Deep Learning Model — Transformer From Scratch

Architecture: A Transformer encoder was implemented entirely from scratch in PyTorch. The model first converts token IDs into 256-dimensional embeddings and adds sinusoidal positional encodings to retain information about token order. The representations are then passed through four Transformer encoder

layers. Each layer consists of multi-head self-attention followed by residual connections and layer normalization, followed by a position-wise feed-forward network with GELU activation, another residual connection, and layer normalization. After the encoder layers, masked mean pooling produces a fixed-size representation using only valid tokens, which is passed through a fully connected classification head to predict whether each candidate answer is correct.

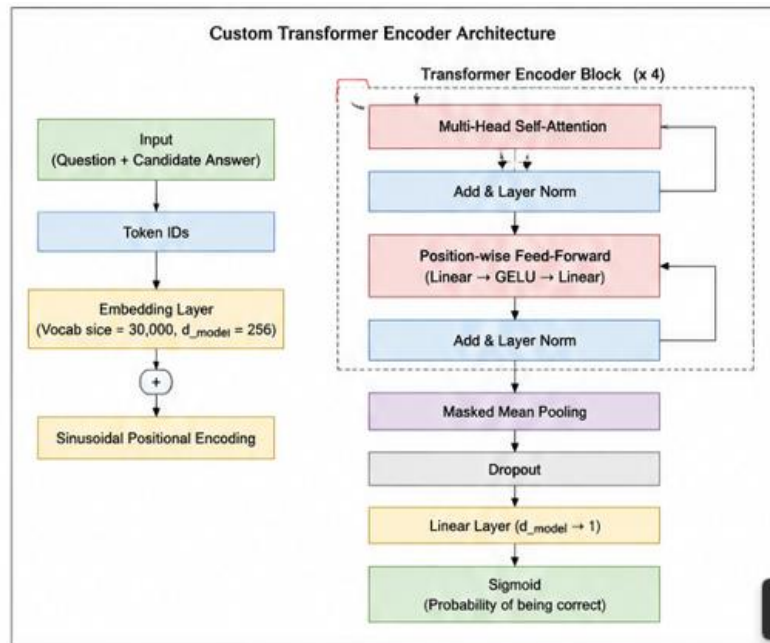


Figure — Architecture of the custom from-scratch Transformer classifier.

Salient points. The main advantage of this architecture is that it demonstrates the core Transformer components without relying on a pretrained Transformer checkpoint. Self-attention allows every token in the question–candidate pair to interact with other tokens, while positional encoding provides information about their order. Residual connections and layer normalization help stabilize training, and masked mean pooling ensures that padding tokens do not affect the final representation. The model treats each question–candidate pair as a binary classification problem, producing a probability that the candidate answer is correct. The five candidate probabilities are then ranked to obtain the top three predictions.

Training strategy. The model used an embedding dimension of 256, 8 attention heads, 4 Transformer layers, a feed-forward dimension of 512, dropout of 0.2, batch size 32, and AdamW optimization. A cosine annealing learning-rate scheduler was used, with gradient clipping at a maximum norm of 1.0 and early stopping based on validation MAP@3. Five-fold cross-validation was used to evaluate the model.

5.3 Model 3: DeBERTa-v3-small

Architecture. DeBERTa-v3-small is a pretrained Transformer encoder developed by Microsoft. DeBERTa extends the BERT architecture using disentangled attention, where token content and positional information are represented separately, allowing the model to model both semantic content and relative position more effectively. The model was loaded using Hugging Face's microsoft/deberta-v3-small checkpoint and fine-tuned for the MCQ task.

Salient points. DeBERTa is suitable for this task because its self-attention mechanism allows the model to capture contextual relationships between the question and candidate answer. Its pretrained representations provide a strong starting point compared with training a Transformer entirely from scratch. The model processes the question and candidate answer as a text pair, and the resulting scores are used to rank the five candidate options.

Fine-tuning strategy. The DeBERTa-v3-small model was fine-tuned using a pretrained Hugging Face tokenizer and PyTorch-based training pipeline. The maximum sequence length was set to 256 tokens, batch size to 16, learning rate to 1×10^{-5} , and training was performed for 4 epochs. The question and candidate answer were tokenized as a pair, padded or truncated to the selected sequence length, and accompanied by an attention mask. The model parameters were fine-tuned for the downstream MCQ task rather than using DeBERTa only as a frozen feature extractor. No quantization was applied.

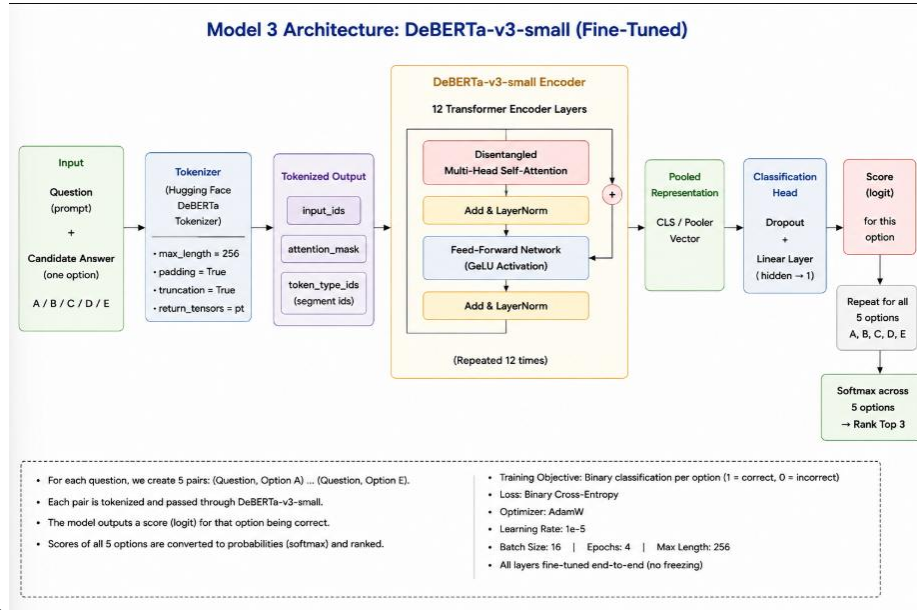


Figure — Architecture of the pretrained DeBERTa Model.

6. Performance & Comparative Analysis

6.1 Evaluation metrics. The primary evaluation metric was Mean Average Precision at 3 (MAP@3), as specified by the Kaggle competition. For each question, the model ranks the five candidate answers and receives a score of 1 if the correct answer is ranked first, 1/2 if ranked second, 1/3 if ranked third, and 0 if it is not present in the top three. The final MAP@3 is the mean of these scores across all questions. Accuracy and macro F1-score were also monitored during model training to evaluate the correctness of the predicted top-1 answer and the balance of classification performance across the five answer classes..

6.2 Training performance. The three Transformer-based models showed different validation performance. The custom Transformer achieved the highest local validation performance, with a 5-fold MAP@3 of 0.9947. DeBERTa-v3-small achieved a MAP@3 of 0.9556 on an 85/15 validation split, while ELECTRA achieved 0.673 using GroupKFold, which provided a stricter evaluation by grouping questions with the same option sets. Training loss, validation accuracy, validation F1-score, MAP@3, and learning rate were logged to Weights & Biases for the experiments. The W&B plots were used to monitor convergence, compare experiments with different learning rates, identify changes in validation performance across epochs, and observe whether the models were continuing to improve or beginning to overfit.

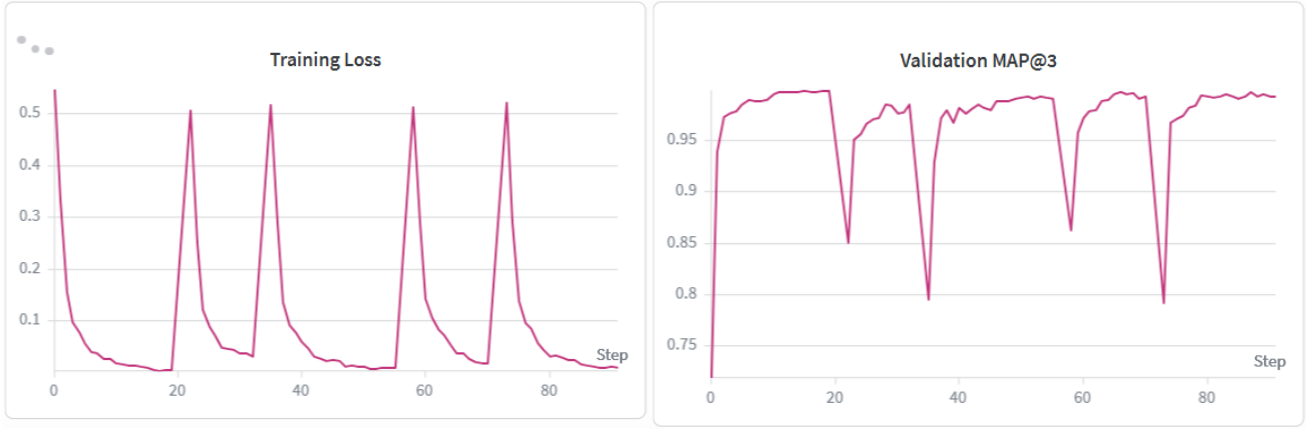


Fig: Transformer model Validation MAP@3 and training loss

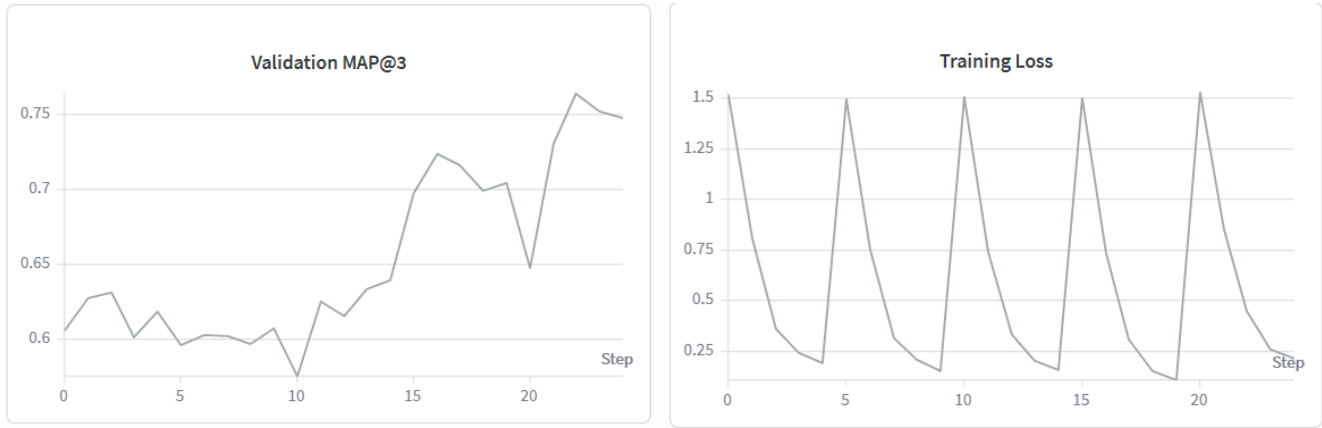


Fig. ELECTRA model Validation MAP@3 and training loss

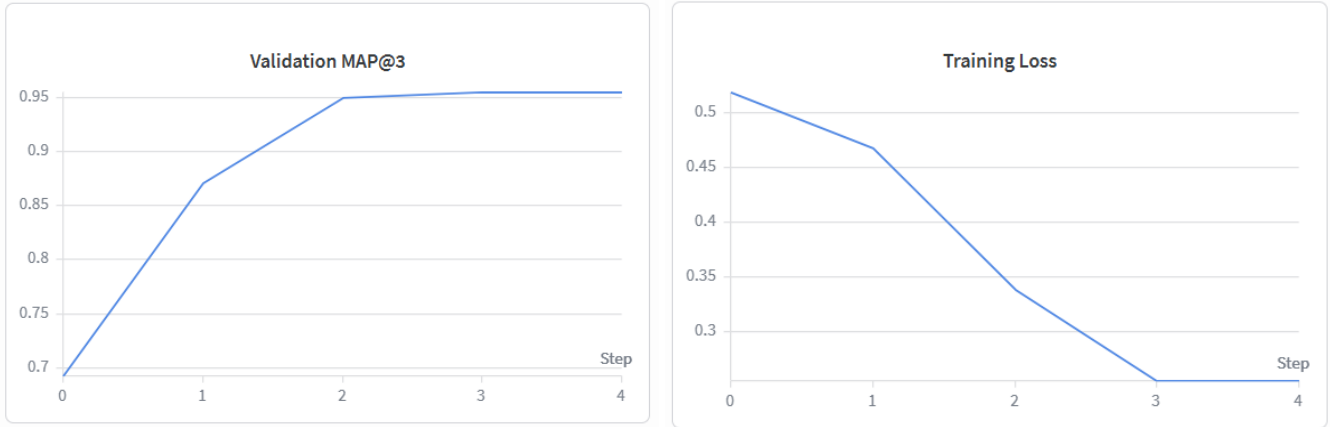


Fig. DeBERTa model Validation MAP@3 and training loss

6.3 Comparative Report

Model	Val. MAP@3	Validation Method
TF-IDF + Logistic Regression	Not computed*	Hold-out (80/20 stratified)
Transformer (from scratch)	0.9947 (± 0.0047)	5-fold Stratified CV, split by question id
ELECTRA-base	0.6730	5-fold GroupKFold (leakage-safe)

DeBERTa-v3-small	0.9556	Hold-out (85/15, by question id)
------------------	--------	----------------------------------

6.4 Kaggle Performance

Final leaderboard score: 0.75893 (MAP@3); rank 237. The submission is an 80%/20% weighted blend of ELECTRA and calibrated from-scratch-Transformer probabilities. This score sits well below the from-scratch Transformer's own validation number (0.9947) and DeBERTa's (0.9556), but above ELECTRA's conservative GroupKFold estimate (0.673).

7. Conclusion & Future Work

7.1 Key learnings.

- Validation design matters as much as model choice — three very different MAP@3 numbers (0.673 / 0.956 / 0.995) on the same data traced mainly to how leakage-resistant each split was.
- Implementing a Transformer from scratch built real intuition for attention, residual connections, and pooling that is easy to skip when only calling a pretrained API.
- Fine-tuning pretrained models needs different instincts than training from scratch — a ~20-30x smaller learning rate and warmup were essential to avoid destroying pretrained knowledge.

7.2 Challenges faced.

- Recognising that question-id-based splitting alone does not prevent leakage from paraphrased duplicate questions — only found via dedicated duplicate analysis .
- GPU memory limits when fine-tuning ELECTRA with 5 options per example, resolved with a small batch size plus gradient accumulation.
- Explaining the local-validation-vs-leaderboard gap, traced to differing leakage-sensitivity across validation protocols.

7.3 Areas for improvement.

- Re-validate all three models with one consistent, leakage-safe GroupKFold protocol before choosing ensemble weights.
- Include DeBERTa in the final weighted ensemble and tune blend weights against a leakage-safe score rather than a fixed guess.
- Try a larger pretrained backbone (e.g. deberta-v3-base) and explore stacking a small meta-model on out-of-fold predictions instead of manual weighting.

8. References

1. Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
2. Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). ELECTRA. ICLR 2020. <https://arxiv.org/abs/2003.10555>
3. He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa. ICLR 2021. <https://arxiv.org/abs/2006.03654>
4. HuggingFace model cards (google/electra-base-discriminator; microsoft/deberta-v3-small) & transformers docs — <https://huggingface.co>. Loshchilov & Hutter (2019), AdamW, ICLR 2019 — <https://arxiv.org/abs/1711.05101>. PyTorch (<https://pytorch.org>), scikit-learn (<https://scikit-learn.org>), Weights & Biases docs (<https://docs.wandb.ai>); Kaggle — Smart MCQ Solver Challenge (competition page & dataset).